Assignment-17

Hallticket no:2303A510A6

Batch no:02

Datapreprocessing

Task 1: Perform EDA on Penguin dataset(from seaborn)

- Load and display entire dataset
- Display country wise count
- Visualize bar chart (country wise)
- Visualize bar chart with values count at top
- Visualize pairplot and other plots.

Code:

```python
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

# Load the penguins dataset
penguins_df = sns.load_dataset('penguins')

print("First 5 rows of the dataset:")
display(penguins_df.head())
```

```
First 5 rows of the dataset:
```

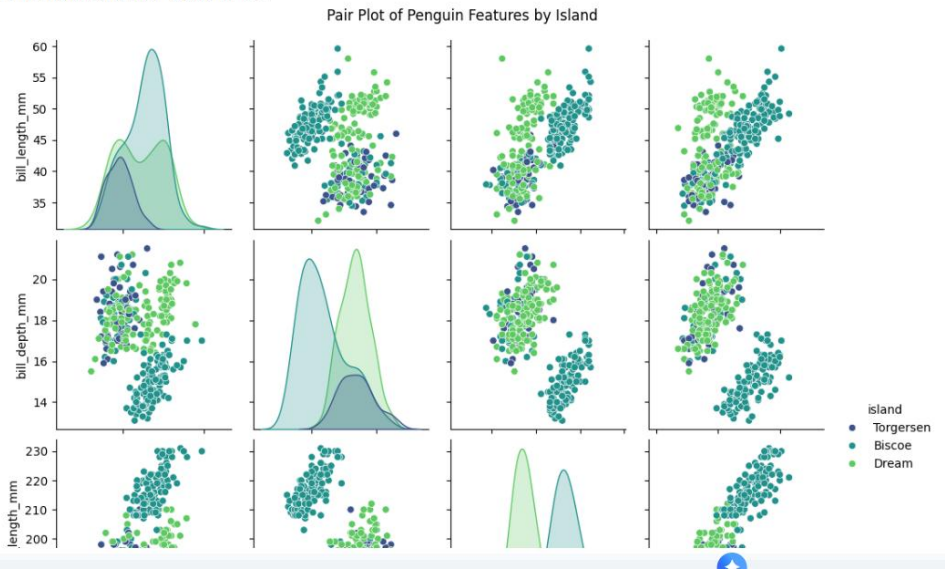
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

```python
print("\nInformation about the dataset:")
penguins_df.info()
```

```
Information about the dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    object
 1   island             344 non-null    object
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

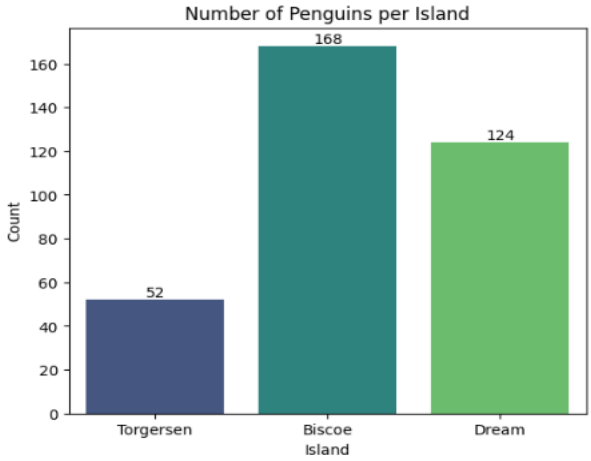
```python
print("\nVisualizing pairplot with 'island' as hue:")
sns.pairplot(penguins_df.dropna(), hue='island', palette='viridis')
plt.suptitle('Pair Plot of Penguin Features by Island', y=1.02)
plt.show()
```

```
Visualizing pairplot with 'island' as hue:
```

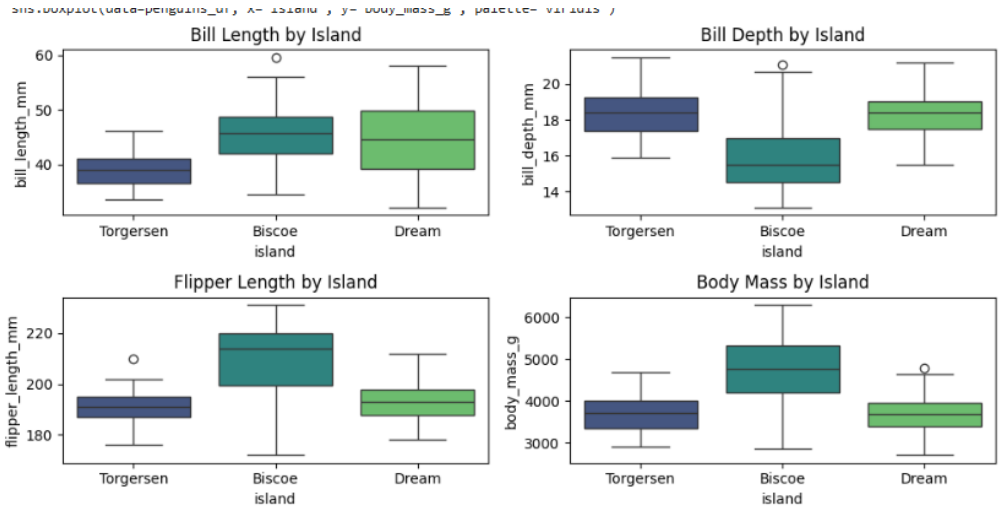


```python
print("\nNumber of penguins per island with value counts:")
plt.figure(figsize=(6, 5))
sns.countplot(data=penguins_df, x='island', palette='viridis', legend=False)
plt.title('Number of Penguins per Island')
plt.xlabel('Island')
plt.ylabel('Count')

# Add value counts on top of each bar
ax = plt.gca()
for container in ax.containers:
    ax.bar_label(container)

plt.show()
```

```
Number of penguins per island with value counts:
/tmp/ipykernel_5170/1566351922.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and

  sns.countplot(data=penguins_df, x='island', palette='viridis', legend=False)
```




```python
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
sns.boxplot(data=penguins_df, x='island', y='bill_length_mm', palette='viridis')
plt.title('Bill Length by Island')

plt.subplot(2, 2, 2)
sns.boxplot(data=penguins_df, x='island', y='bill_depth_mm', palette='viridis')
plt.title('Bill Depth by Island')

plt.subplot(2, 2, 3)
sns.boxplot(data=penguins_df, x='island', y='flipper_length_mm', palette='viridis')
plt.title('Flipper Length by Island')

plt.subplot(2, 2, 4)
sns.boxplot(data=penguins_df, x='island', y='body_mass_g', palette='viridis')
plt.title('Body Mass by Island')

plt.tight_layout()
```



Task 2: Perform EDA on HealthExp dataset(seaborn)

- Load and display entire dataset
- Display columns and statistical summary(describe)
- Display missing values , and draw bar chart
- Visualize pairplot, regplot and box plot, scatter plot etc

Code:

```python
import seaborn as sns
import pandas as pd

# Load the healthexp dataset
health_exp = sns.load_dataset('healthexp')

# Display the first 5 rows of the dataset
display(health_exp.head())
```

	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

```python
# Get a concise summary of the DataFrame (data types, non-null values)
display(health_exp.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 274 entries, 0 to 273
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Year             274 non-null    int64
 1   Country          274 non-null    object
 2   Spending_USD     274 non-null    float64
 3   Life_Expectancy  274 non-null    float64
dtypes: float64(2), int64(1), object(1)
memory usage: 8.7+ KB
None
```

```python
# Display descriptive statistics for numerical columns
display(health_exp.describe())
```

	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

```python
# Display the column names of the health_exp dataset
print("Columns in health_exp dataset:")
print(health_exp.columns.tolist())
```

```
Columns in health_exp dataset:
['Year', 'Country', 'Spending_USD', 'Life_Expectancy']
```

```python
# Display the statistical summary for the health_exp dataset
display(health_exp.describe())
```

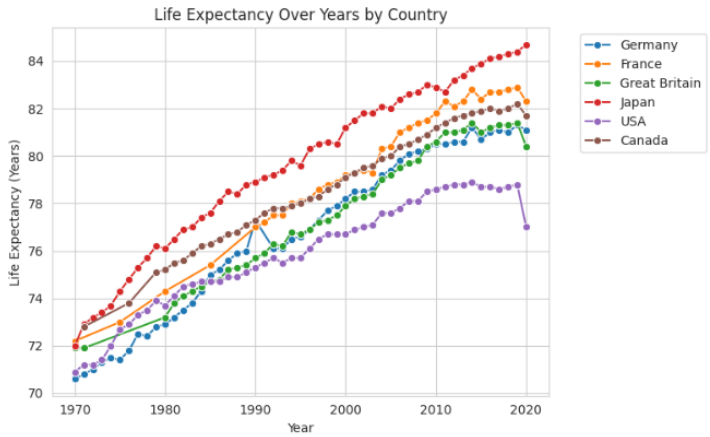
	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000

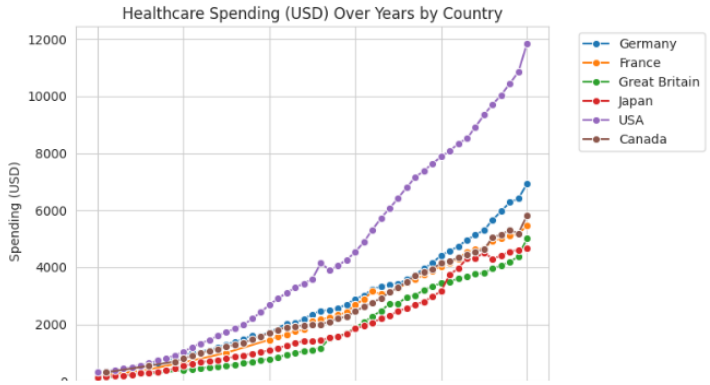
```python
# Check unique countries in the 'Country' column
print("Unique Countries:")
print(health_exp['Country'].unique())
print(f"Number of unique countries: {health_exp['Country'].nunique()}")
```

```
Unique Countries:
['Germany' 'France' 'Great Britain' 'Japan' 'USA' 'Canada']
Number of unique countries: 6
```

Now, let's visualize the trends of `Life_Expectancy` and `Spending_USD` over the years for these countries using line plots.

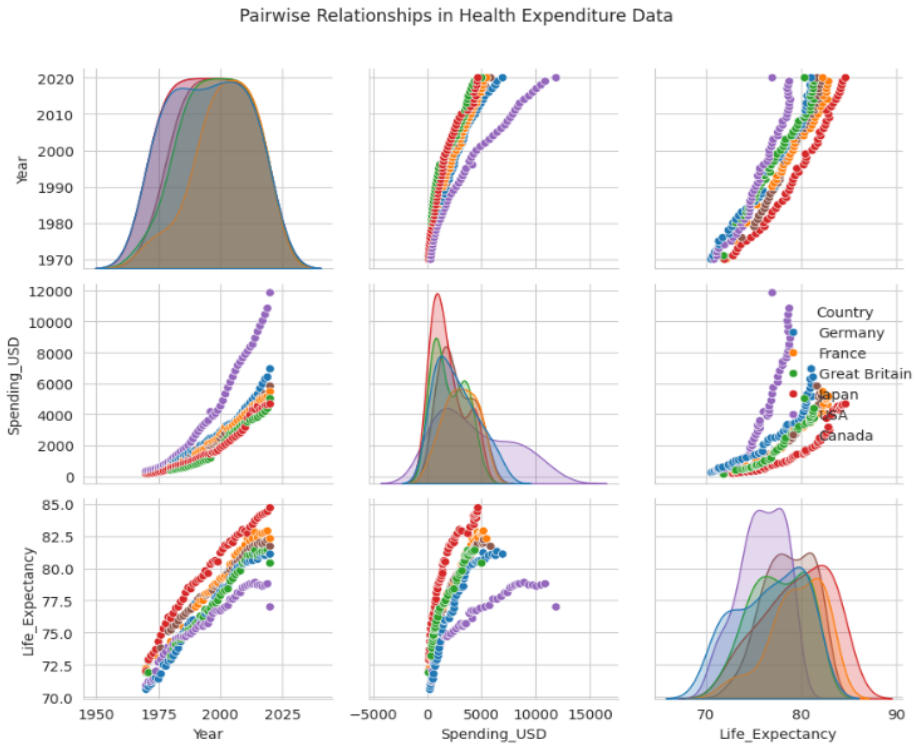
```python
import matplotlib.pyplot as plt

# Set the aesthetic style of the plots
sns.set_style("whitegrid")

# Plot Life_Expectancy over Year for each Country
plt.figure(figsize=(8, 5))
sns.lineplot(data=health_exp, x='Year', y='Life_Expectancy', hue='Country', marker='o')
plt.title('Life Expectancy Over Years by Country')
plt.xlabel('Year')
plt.ylabel('Life Expectancy (Years)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.show()

# Plot Spending_USD over Year for each Country
plt.figure(figsize=(8, 5))
sns.lineplot(data=health_exp, x='Year', y='Spending_USD', hue='Country', marker='o')
plt.title('Healthcare Spending (USD) Over Years by Country')
plt.xlabel('Year')
plt.ylabel('Spending (USD)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.show()
```





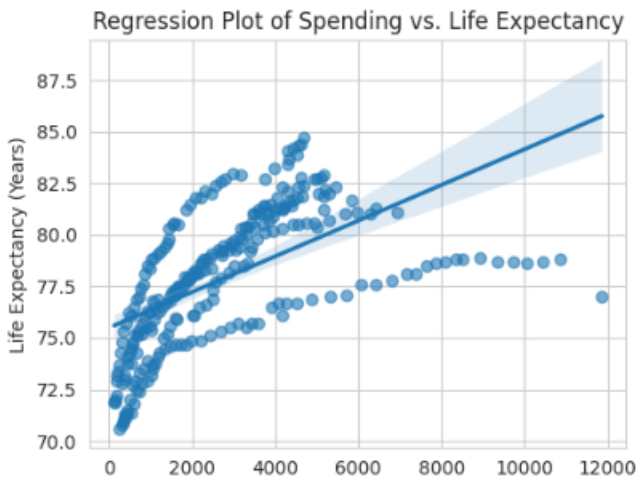
```python
# Pairplot to visualize pairwise relationships
print("Generating Pairplot...")
sns.pairplot(health_exp, hue='Country', diag_kind='kde')
plt.suptitle('Pairwise Relationships in Health Expenditure Data', y=1.02)
plt.tight_layout()
plt.show()
```

```
Generating Pairplot...
```



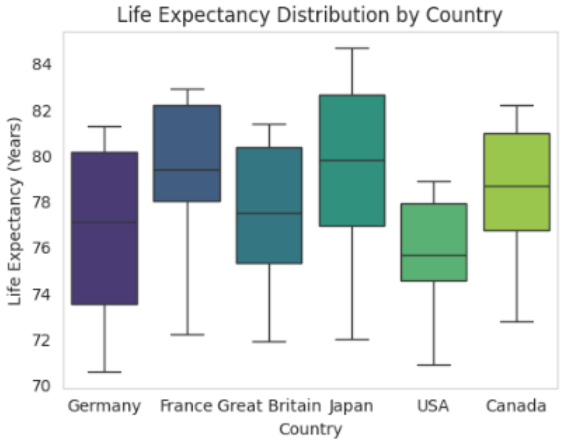

```python
# Regression plot for Spending_USD vs Life_Expectancy
print("Generating Regplot...")
plt.figure(figsize=(5, 4))
sns.regplot(data=health_exp, x='Spending_USD', y='Life_Expectancy', scatter_kws={'alpha':0.6})
plt.title('Regression Plot of Spending vs. Life Expectancy')
plt.xlabel('Healthcare Spending (USD)')
plt.ylabel('Life Expectancy (Years)')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Generating Regplot...
```



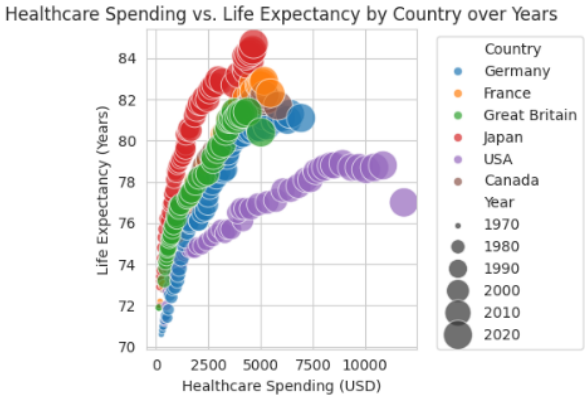
```python
# Boxplot of Life_Expectancy by Country
print("Generating Boxplot...")
plt.figure(figsize=(5, 4))
sns.boxplot(data=health_exp, x='Country', y='Life_Expectancy', palette='viridis')
plt.title('Life Expectancy Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Life Expectancy (Years)')
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

```
Generating Boxplot...
/tmp/ipykernel_7698/2463886276.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` vari

  sns.boxplot(data=health_exp, x='Country', y='Life_Expectancy', palette='viridis')
```




```python
# Scatter plot of Spending_USD vs Life_Expectancy with Country as hue
print("Generating Scatter Plot...")
plt.figure(figsize=(5, 4))
sns.scatterplot(data=health_exp, x='Spending_USD', y='Life_Expectancy', hue='Country', size='Year', sizes=(20, 400), alpha=0.7)
plt.title('Healthcare Spending vs. Life Expectancy by Country over Years')
plt.xlabel('Healthcare Spending (USD)')
plt.ylabel('Life Expectancy (Years)')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Generating Scatter Plot...
```



Task 3: Perform following task on given dataset (Train the model)

• Analyze the Planets dataset to understand its structure, features, and missing values.

• Perform data cleaning and preprocessing to handle missing or incorrect data.

• Train the dataset using different machine learning models to make predictions.

• Compare and explain the performance of all trained models with proper reasoning.

Code:

```
import pandas as pd
import numpy as np

# Create a dummy 'Planets dataset'
np.random.seed(42) # for reproducibility
dummy_data_planets = {
    'planet_name': ['Earth', 'Mars', 'Jupiter', 'Saturn', 'Neptune', 'Venus', 'Uranus', 'Mercury', 'Pluto', 'Kepler-186f'],
    'radius_km': np.random.normal(60000, 20000, 10),
    'mass_earths': np.random.normal(100, 50, 10),
    'has_rings': [False, False, True, True, True, False, True, False, False, None],
    'orbital_period_days': np.random.normal(365, 200, 10),
    'num_moons': np.random.randint(0, 80, 10).astype(float), # Convert to float to allow NaN
    'atmosphere_composition': ['Nitrogen, Oxygen', 'CO2', 'Hydrogen, Helium', 'Hydrogen, Helium', 'Hydrogen, Helium', 'CO2', 'Hydrogen, Helium', 'None', 'Nitroge
}

# Introduce some NaNs to simulate missing values
dummy_data_planets['radius_km'][2] = np.nan # Jupiter's radius
dummy_data_planets['mass_earths'][5] = np.nan # Venus's mass
dummy_data_planets['num_moons'][8] = np.nan # Pluto's moons

dummy_planets_df = pd.DataFrame(dummy_data_planets)

# Add one more row to make it 11, matching the length of planet_name
dummy_planets_df.loc[10] = ['Exoplanet-X', 15000, 2.5, False, 120, 0, 'Unknown']

print("Dummy Planets Dataset created:")
display(dummy_planets_df.head())
```

```
Dummy Planets Dataset created:
```

	planet_name	radius_km	mass_earths	has_rings	orbital_period_days	num_moons	atmosphere_composition
0	Earth	69934.283060	76.829115	False	658.129754	59.0	Nitrogen, Oxygen
1	Mars	57234.713977	76.713512	False	319.844740	70.0	CO2
2	Jupiter	NaN	112.098114	True	378.505641	43.0	Hydrogen, Helium
3	Saturn	90460.597128	4.335988	True	80.050363	7.0	Hydrogen, Helium
4	Neptune	55316.932506	13.754108	True	256.123455	46.0	Hydrogen, Helium

Dataset Structure and Features

```python
# Display the DataFrame's information to see its structure, column names, and data types
print("DataFrame Info:")
dummy_planets_df.info()

print("\nDataFrame Columns (Features):")
print(dummy_planets_df.columns.tolist())
```

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 11 entries, 0 to 10
Data columns (total 7 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   planet_name             11 non-null     object
 1   radius_km               10 non-null     float64
 2   mass_earths             10 non-null     float64
 3   has_rings               10 non-null     object
 4   orbital_period_days     11 non-null     float64
 5   num_moons               10 non-null     float64
 6   atmosphere_composition  11 non-null     object
dtypes: float64(4), object(3)
memory usage: 704.0+ bytes

DataFrame Columns (Features):
['planet_name', 'radius_km', 'mass_earths', 'has_rings', 'orbital_period_days', 'num_moons', 'atmosphere_composition']
```

```python
# Identify missing values in each column
print("Missing values per column:")
display(dummy_planets_df.isnull().sum())

# Calculate the percentage of missing values
print("\nPercentage of missing values per column:")
display(dummy_planets_df.isnull().sum() / len(dummy_planets_df) * 100)
```

```
Missing values per column:
```

	0
planet_name	0
radius_km	1
mass_earths	1
has_rings	1
orbital_period_days	0
num_moons	1
atmosphere_composition	0

dtype: int64

Data Cleaning and Preprocessing

+ Code + Text

Handling Missing Values

```python
# Impute missing numerical values with the median
# For radius_km and mass_earths
for col in ['radius_km', 'mass_earths', 'num_moons']:
    if dummy_planets_df[col].isnull().any():
        median_val = dummy_planets_df[col].median()
        dummy_planets_df[col].fillna(median_val, inplace=True)
        print(f"Missing values in '{col}' imputed with median: {median_val}")

# Impute missing 'has_rings' (categorical/boolean) with the mode
if dummy_planets_df['has_rings'].isnull().any():
    mode_val = dummy_planets_df['has_rings'].mode()[0] # .mode() returns a Series, take the first element
    dummy_planets_df['has_rings'].fillna(mode_val, inplace=True)
    print(f"Missing values in 'has_rings' imputed with mode: {mode_val}")

print("\nMissing values after imputation:")
display(dummy_planets_df.isnull().sum())
```

```
Missing values in 'radius_km' imputed with median: 63584.498518400476
Missing values in 'mass_earths' imputed with median: 51.97862010360913
Missing values in 'num_moons' imputed with median: 39.0
Missing values in 'has_rings' imputed with mode: False

Missing values after imputation:
/tmp/ipykernel_10741/1013560125.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perfor


  dummy_planets_df[col].fillna(median_val, inplace=True)
/tmp/ipykernel_10741/1013560125.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace metho
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perfor


  dummy_planets_df['has_rings'].fillna(mode_val, inplace=True)
/tmp/ipykernel_10741/1013560125.py:12: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Ca
  dummy_planets_df['has_rings'].fillna(mode_val, inplace=True)
```

	0
planet_name	0

Removing Duplicate Rows

```python
initial_rows = len(dummy_planets_df)
dummy_planets_df.drop_duplicates(inplace=True)
removed_rows = initial_rows - len(dummy_planets_df)

if removed_rows > 0:
    print(f"Removed {removed_rows} duplicate rows.")
else:
    print("No duplicate rows found.")

print("\nDataFrame after removing duplicates:")
display(dummy_planets_df.head())
```

```
No duplicate rows found.

DataFrame after removing duplicates:
```

	planet_name	radius_km	mass_earths	has_rings	orbital_period_days	num_moons	atmosphere_composition
0	Earth	69934.283060	76.829115	False	658.129754	59.0	Nitrogen, Oxygen
1	Mars	57234.713977	76.713512	False	319.844740	70.0	CO2
2	Jupiter	63584.498518	112.098114	True	378.505641	43.0	Hydrogen, Helium
3	Saturn	90460.597128	4.335988	True	80.050363	7.0	Hydrogen, Helium
4	Neptune	55316.932506	13.754108	True	256.123455	46.0	Hydrogen, Helium

2. Decision Tree Classifier

[21]
✓ 0s

```python
# Initialize and train the Decision Tree Classifier
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_dt = dt_model.predict(X_test)

# Evaluate the model
print("Decision Tree Classifier Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_dt):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_dt))
```

```
Decision Tree Classifier Performance:
Accuracy: 1.0000
Classification Report:
              precision    recall  f1-score   support

       False       1.00      1.00      1.00         3
        True       1.00      1.00      1.00         1

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4
```

3. Random Forest Classifier

[22]
✓ 0s

```python
# Initialize and train the Random Forest Classifier
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the model
print("Random Forest Classifier Performance:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print("Classification Report:")
print(classification_report(y_test, y_pred_rf))
```

```
Random Forest Classifier Performance:
```

```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Define features (X) and target (y)
# 'planet_name' is an identifier and should not be used as a feature
X = dummy_planets_df.drop(columns=['planet_name', 'has_rings'])
y = dummy_planets_df['has_rings']

print("Features (X) head:")
display(X.head())
print("\nTarget (y) head:")
display(y.head())

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

print(f"\nTraining features shape: {X_train.shape}")
print(f"Testing features shape: {X_test.shape}")
print(f"Training target shape: {y_train.shape}")
print(f"Testing target shape: {y_test.shape}")
```

```
Features (X) head:
```

	radius_km	mass_earths	orbital_period_days	num_moons	atm_CO2	atm_Hydrogen, Helium	atm_Nitrogen	atm_Nitrogen, Oxygen	atm_None	atm_Unknown
0	69934.283060	76.829115	658.129754	59.0	False	False	False	True	False	False
1	57234.713977	76.713512	319.844740	70.0	True	False	False	False	False	False
2	63584.498518	112.098114	378.505641	43.0	False	True	False	False	False	False
3	90460.597128	4.335988	80.050363	7.0	False	True	False	False	False	False
4	55316.932506	13.754108	256.123455	46.0	False	True	False	False	False	False

```python
# One-hot encode 'atmosphere_composition'
# 'planet_name' is typically an identifier and might not be directly encoded for modeling
dummy_planets_df = pd.get_dummies(dummy_planets_df, columns=['atmosphere_composition'], prefix='atm')
print("One-hot encoded 'atmosphere_composition'.")

print("\nDataFrame after categorical conversion:")
display(dummy_planets_df.head())
print("\nUpdated DataFrame Info:")
dummy_planets_df.info()
```

```
Converted 'has_rings' to boolean type.
One-hot encoded 'atmosphere_composition'.

DataFrame after categorical conversion:
```

	planet_name	radius_km	mass_earths	has_rings	orbital_period_days	num_moons	atm_CO2	atm_Hydrogen, Helium	atm_Nitrogen	atm_Nitrogen, Oxygen	atm_None	atm_Unknown
0	Earth	69934.283060	76.829115	False	658.129754	59.0	False	False	False	True	False	False
1	Mars	57234.713977	76.713512	False	319.844740	70.0	True	False	False	False	False	False
2	Jupiter	63584.498518	112.098114	True	378.505641	43.0	False	True	False	False	False	False
3	Saturn	90460.597128	4.335988	True	80.050363	7.0	False	True	False	False	False	False
4	Neptune	55316.932506	13.754108	True	256.123455	46.0	False	True	False	False	False	False

```
Updated DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 11 entries, 0 to 10
Data columns (total 12 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   planet_name            11 non-null     object
 1   radius_km              11 non-null     float64
 2   mass_earths            11 non-null     float64
 3   has_rings              11 non-null     bool
 4   orbital_period_days    11 non-null     float64
 5   num_moons              11 non-null     float64
 6   atm_CO2                11 non-null     bool
 7   atm_Hydrogen, Helium   11 non-null     bool
 8   atm_Nitrogen           11 non-null     bool
 9   atm_Nitrogen, Oxygen   11 non-null     bool
 10  atm_None               11 non-null     bool
 11  atm_Unknown            11 non-null     bool
```

Task 4: Perform following task on given dataset (Train the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design a EfficientNet model for imageclassification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.
- Visualize the training history (accuracy and loss curves).

Code:

```python
import torch
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Define a transformation to convert images to tensors
transform = transforms.Compose([
    transforms.ToTensor()
])

# Load the MNIST training dataset
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)

# Display a sample image
# Get one image and its label
image, label = train_dataset[0]

# Convert the tensor to a NumPy array for displaying with matplotlib
# Permute the dimensions if necessary (e.g., from C, H, W to H, W, C for color images, but MNIST is graysc
# For grayscale, squeeze removes the channel dimension (1, 28, 28) -> (28, 28)
plt.imshow(image.squeeze().numpy(), cmap='gray')
plt.title(f'Label: {label}')
plt.axis('off') # Hide axes
plt.show()
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 22.8MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 607kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 5.68MB/s]
```

```python
print(f"Number of training samples: {len(train_dataset)}")
print(f"Number of classes: {len(train_dataset.classes)}")
print(f"Class names: {train_dataset.classes}")
```

```
Number of training samples: 60000
Number of classes: 10
Class names: ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 - six', '7 - seven', '8 -
```

Data Processing: Normalization and Data Loaders

EfficientNet Model for Image Classification

```python
import torch.nn as nn
import torchvision.models as models

# Check for GPU availability and set the device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cpu
```

```python
# Load a pre-trained EfficientNet B0 model
# EfficientNet B0 is a good starting point, balance of accuracy and computational cost
model = models.efficientnet_b0(pretrained=True)

# Modify the classifier head for MNIST (10 classes)
# EfficientNet's classifier is typically a Linear layer following an AdaptiveAvgPool2d
# The name of the classifier head depends on the EfficientNet version.
# For EfficientNet_B0, it's model.classifier[1] as per torchvision implementation.
num_ftrs = model.classifier[1].in_features
model.classifier[1] = nn.Linear(num_ftrs, 10) # 10 classes for MNIST (digits 0-9)

# Move the model to the chosen device
model = model.to(device)

print("EfficientNet B0 model loaded and classifier head modified for 10 classes.")
print(model)
```

```
kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
2, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
True)


ial(
mActivation(
32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), groups=32, bias=False)
rm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
place=True)

citation(
daptiveAvgPool2d(output_size=1)
d(32, 8, kernel_size=(1, 1), stride=(1, 1))
d(8, 32, kernel_size=(1, 1), stride=(1, 1))
: SiLU(inplace=True)
ation): Sigmoid()
```

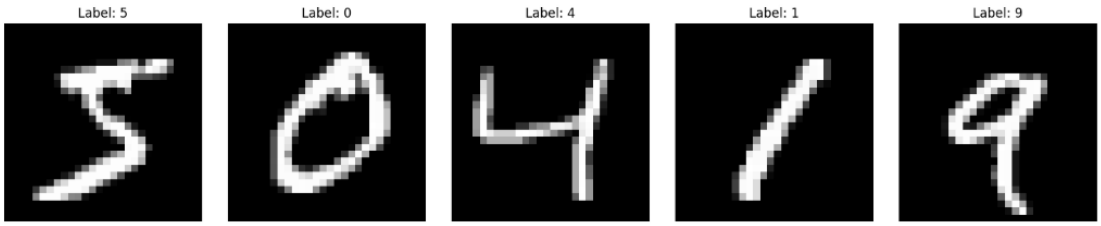
```python
# Define Loss Function and Optimizer
criterion = nn.CrossEntropyLoss() # Suitable for multi-class classification
optimizer = torch.optim.Adam(model.parameters(), lr=0.001) # Adam optimizer with a learning rate

# Training parameters
num_epochs = 5 # You can adjust this number

print(f"Loss function: {criterion}")
print(f"Optimizer: {optimizer.__class__.__name__}")
print(f"Number of epochs set to: {num_epochs}")
```

```
Loss function: CrossEntropyLoss()
Optimizer: Adam
Number of epochs set to: 5
```

```python
# Define transformations: convert to tensor and normalize
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,)) # Mean and Std for MNIST
])

# Load the MNIST training and test datasets with the defined transformations
train_dataset_processed = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=tra
test_dataset = torchvision.datasets.MNIST(root='./data', train=False, download=True, transform=transform)

print(f"Number of processed training samples: {len(train_dataset_processed)}")
print(f"Number of test samples: {len(test_dataset)}")
```

```
Number of processed training samples: 60000
```

```python
fig, axes = plt.subplots(1, 5, figsize=(15, 3))

for i, ax in enumerate(axes):
    # Get an image and its label
    image, label = train_dataset[i]

    # Display the image
    ax.imshow(image.squeeze().numpy(), cmap='gray')
    ax.set_title(f'Label: {label}')
    ax.axis('off') # Hide axes

plt.tight_layout()
plt.show()
```



Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.

• Standardize text values (e.g., city names in lowercase).

• Encode categorical variables (gender, city).

Sample Code:

import pandas as pd

data = pd.read_csv("shopping_trends.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

• A cleaned Pandas data frame ready for analysis

Code:

```python
import pandas as pd
import zipfile
import io

# Load the dataset
try:
    zip_file_path = '/content/archive (4).zip'
    csv_file_name = 'shopping_trends.csv'

    # Open the zip file
    with zipfile.ZipFile(zip_file_path, 'r') as z:
        # Read the specific CSV file from the zip archive into a BytesIO object
        with z.open(csv_file_name) as f:
            df = pd.read_csv(io.TextIOWrapper(f, 'utf-8'))

    print("Dataset loaded successfully!")
except FileNotFoundError:
    print(f"Error: '{zip_file_path}' or '{csv_file_name}' not found. Please upload the file to your Colab environment.")
    df = None
except KeyError:
    print(f"Error: '{csv_file_name}' not found inside the zip file '{zip_file_path}'.")
    df = None
except Exception as e:
    print(f"Error loading dataset: {e}")
    df = None

if df is not None:
    # Display the first 5 rows of the DataFrame
    print("\nFirst 5 rows of the dataset:")
    display(df.head())

    # Display basic information about the DataFrame (data types, non-null values)
    print("\nDataFrame Info:")
    df.info()
```

```
Dataset loaded successfully!

First 5 rows of the dataset:
```

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color	Season	Review Rating	Subscription Status	Payment Method	Shipping Type	Disco Appl
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray	Winter	3.1	Yes	Credit Card	Express	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon	Winter	3.1	Yes	Bank Transfer	Express	
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon	Spring	3.1	Yes	Cash	Free Shipping	

```python
    print("\nUnique values in 'Location' before standardization:")
    display(df['Location'].nunique())
    display(df['Location'].value_counts().head())

    # Standardize 'Location' column to lowercase
    df['Location'] = df['Location'].str.lower()

    print("\nUnique values in 'Location' after standardization:")
    display(df['Location'].nunique())
    display(df['Location'].value_counts().head())

    print("\nText standardization applied to 'Location' column. You can apply similar transformations to other relevant text columns if n
```

```
Unique values in 'Location' before standardization:
50
```

Location	count
Montana	96
California	95
Idaho	93
Illinois	92
Alabama	89

dtype: int64

```
Unique values in 'Location' after standardization:
50
```

Location	count
montana	96
california	95
idaho	93
illinois	92
alabama	89

dtype: int64

```python
if df is not None:
    # Check for missing values
    print("\nMissing values per column:")
    display(df.isnull().sum())

    # Check for duplicate rows
    print("\nNumber of duplicate rows:")
    num_duplicates = df.duplicated().sum()
    print(f"Total duplicate rows: {num_duplicates}")

    if num_duplicates > 0:
        print("Removing duplicate rows...")
        df.drop_duplicates(inplace=True)
        print(f"Remaining rows after removing duplicates: {len(df)}")
    else:
        print("No duplicate rows found.")
```

```
Missing values per column:
```

	0
Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	0
Subscription Status	0
Payment Method	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0

```python
# Inspect unique values in categorical columns (object type)
print("\nUnique values in categorical columns:")
for col in df.select_dtypes(include='object').columns:
    print(f"Column '{col}': {df[col].nunique()} unique values")
    if df[col].nunique() < 50: # Display unique values if not too ma
        display(df[col].value_counts())
    else:
        print(" (Too many unique values to display all)")

# Display descriptive statistics for numerical columns
print("\nDescriptive statistics for numerical columns:")
display(df.describe())
```

```
Unique values in categorical columns:
Column 'Gender': 2 unique values
```

Gender	count
Male	2652
Female	1248

dtype: int64

```
Column 'Item Purchased': 25 unique values
```

Item Purchased	count
Blouse	171
Pants	171
Jewelry	171
Shirt	169
Dress	166
Sweater	164
Jacket	163
Coat	161
Sunglasses	161

Task 6 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

• Convert date columns into datetime format.

• Create new features (month, year, day of week).

• Normalize sales amount column.

• Detect and handle outliers in dataset.

Sample Code:

import pandas as pd

data = pd.read_csv("sales_data.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

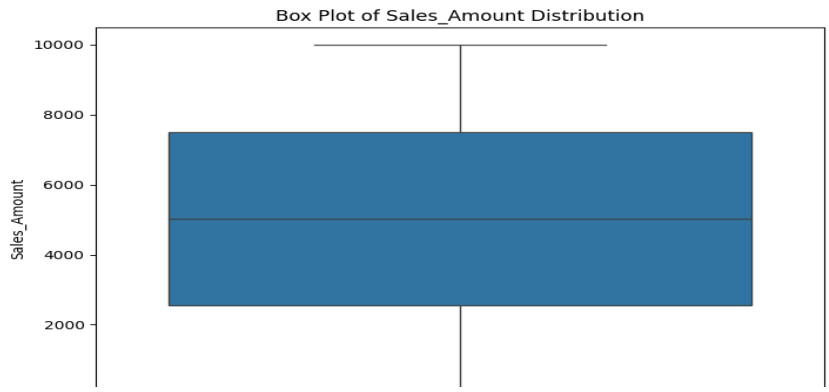
• A transformed dataset with time-based features and normalized values.

Code:

```
import pandas as pd
data = pd.read_csv("/content/archive (5).zip")
print(data.head())
```

```
   Product_ID   Sale_Date Sales_Rep Region  Sales_Amount  Quantity_Sold  \
0        1052  2023-02-03       Bob  North       5053.97             18
1        1093  2023-04-21       Bob   West       4384.02             17
2        1015  2023-09-21     David  South       4631.23             30
3        1072  2023-08-24       Bob  South       2167.94             39
4        1061  2023-03-24   Charlie   East       3750.20             13

  Product_Category  Unit_Cost  Unit_Price Customer_Type  Discount  \
0        Furniture     152.75      267.22     Returning      0.09
1        Furniture    3816.39     4209.44     Returning      0.11
2             Food     261.56      371.40     Returning      0.20
3         Clothing    4330.03     4467.75           New      0.02
4      Electronics     637.37      692.71           New      0.08

  Payment_Method Sales_Channel Region_and_Sales_Rep
0           Cash        Online            North-Bob
1           Cash        Retail             West-Bob
2  Bank Transfer        Retail          South-David
3    Credit Card        Retail            South-Bob
4    Credit Card        Online         East-Charlie
```

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.boxplot(y=data['Sales_Amount'])
plt.title('Box Plot of Sales_Amount Distribution')
plt.ylabel('Sales_Amount')
plt.show()
```



```python
from sklearn.preprocessing import MinMaxScaler

# Initialize the MinMaxScaler
scaler = MinMaxScaler()

# Reshape the 'Sales_Amount' column to a 2D array for scaling
data['Sales_Amount_Normalized'] = scaler.fit_transform(data[['Sales_Amount']])

print("DataFrame with normalized 'Sales_Amount_Normalized' feature:")
display(data.head())

print("\nDescriptive statistics for 'Sales_Amount_Normalized':")
display(data['Sales_Amount_Normalized'].describe())
```

```
DataFrame with normalized 'Sales_Amount_Normalized' feature:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Pr
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	26
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	420
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	37
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	446
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	69

```
Descriptive statistics for 'Sales_Amount_Normalized':
```

	Sales_Amount_Normalized
count	1000.000000
mean	0.497440
std	0.287877
min	0.000000

```python
data['Sale_Month'] = data['Sale_Date'].dt.month
data['Sale_Year'] = data['Sale_Date'].dt.year
data['Sale_DayOfWeek'] = data['Sale_Date'].dt.dayofweek

print("DataFrame with new date features:")
display(data.head())
print("\nDataFrame Info with new features:")
print(data.info())
```

```
DataFrame with new date features:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category	Unit_Cost	Unit_Pr
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture	152.75	26
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture	3816.39	420
2	1015	2023-09-21	David	South	4631.23	30	Food	261.56	37
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing	4330.03	446
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics	637.37	69

```
DataFrame Info with new features:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
```

```
data['Sale_Date'] = pd.to_datetime(data['Sale_Date'])
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Product_ID            1000 non-null   int64
 1   Sale_Date             1000 non-null   datetime64[ns]
 2   Sales_Rep             1000 non-null   object
 3   Region                1000 non-null   object
 4   Sales_Amount          1000 non-null   float64
 5   Quantity_Sold         1000 non-null   int64
 6   Product_Category      1000 non-null   object
 7   Unit_Cost             1000 non-null   float64
 8   Unit_Price            1000 non-null   float64
 9   Customer_Type         1000 non-null   object
 10  Discount              1000 non-null   float64
 11  Payment_Method        1000 non-null   object
 12  Sales_Channel         1000 non-null   object
 13  Region_and_Sales_Rep  1000 non-null   object
dtypes: datetime64[ns](1), float64(4), int64(2), object(7)
memory usage: 109.5+ KB
None
```

Task 7 – Healthcare Data Preparation

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare- risk-factors-dataset

Expected Output:

- A preprocessed dataset ready for machine learning models

```python
import zipfile
import pandas as pd

zip_file_path = '/content/archive (6).zip'
csv_file_name = 'dirty_v3_path.csv' # Corrected CSV file name

with zipfile.ZipFile(zip_file_path, 'r') as zf:
    if csv_file_name in zf.namelist():
        with zf.open(csv_file_name) as f:
            data = pd.read_csv(f)
        print(f"Successfully loaded '{csv_file_name}' from '{zip_file_path}'")
    else:
        print(f"'{csv_file_name}' not found in the zip file. Listing contents:")
        print(zf.namelist())
        raise FileNotFoundError(f"'{csv_file_name}' not found in the zip archive.")

display(data.head())
```

```
Successfully loaded 'dirty_v3_path.csv' from '/content/archive (6).zip'
```

	Age	Gender	Medical Condition	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides
0	46.0	Male	Diabetes	137.04	135.27	28.90	96.04	6	231.88	210.56
1	22.0	Male	Healthy	71.58	113.27	26.29	97.54	2	165.57	129.41
2	50.0	NaN	Asthma	95.24	NaN	22.53	90.31	2	214.94	165.35
3	57.0	NaN	Obesity	NaN	130.53	38.47	96.60	5	197.71	182.13
4	66.0	Female	Hypertension	95.15	178.17	31.12	94.90	4	259.53	115.85

```python
print("Missing values before imputation:")
print(data[['Blood Pressure', 'Cholesterol']].isnull().sum())

# Impute missing 'Blood Pressure' values with the median
median_bp = data['Blood Pressure'].median()
data['Blood Pressure'].fillna(median_bp, inplace=True)
print(f"'Blood Pressure' missing values imputed with median: {median_bp:.2f}")

# Impute missing 'Cholesterol' values with the median
median_cholesterol = data['Cholesterol'].median()
data['Cholesterol'].fillna(median_cholesterol, inplace=True)
print(f"'Cholesterol' missing values imputed with median: {median_cholesterol:.2f}")

print("\nMissing values after imputation:")
print(data[['Blood Pressure', 'Cholesterol']].isnull().sum())
```

```
Missing values before imputation:
Blood Pressure    4500
Cholesterol          0
dtype: int64
'Blood Pressure' missing values imputed with median: 138.32
'Cholesterol' missing values imputed with median: 211.84

Missing values after imputation:
Blood Pressure    0
Cholesterol       0
dtype: int64
/tmp/ipykernel_10741/1523028860.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame c
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=T


  data['Blood Pressure'].fillna(median_bp, inplace=True)
/tmp/ipykernel_10741/1523028860.py:11: FutureWarning: A value is trying to be set on a copy of a DataFrame
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=T
```

```python
from sklearn.preprocessing import MinMaxScaler

# Identify numerical columns (excluding encoded ones and the ones that don't need scaling like boolean flag
numerical_cols = data.select_dtypes(include=['int64', 'float64']).columns.tolist()

# Exclude columns that are already encoded or not relevant for scaling if any (e.g., IDs, or the dummy vari
# For now, let's assume all remaining int/float columns are to be scaled.
# If there are specific columns that should not be scaled, they need to be excluded here.
# For example, if there were 'ID' columns or already scaled columns.

# Check for newly created dummy variables, these are typically boolean (0 or 1) and often not scaled.
# The dtype=int in get_dummies ensures they are int, so they might be picked up here.
# Let's filter out columns that start with Gender_ or Medical Condition_ as these are one-hot encoded.

final_numerical_cols = [col for col in numerical_cols if not (col.startswith('Gender_') or col.startswith('

print("Numerical columns to scale:", final_numerical_cols)

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Apply Min-Max scaling to the identified numerical columns
data[final_numerical_cols] = scaler.fit_transform(data[final_numerical_cols])

print("\nDataFrame after Min-Max scaling of numerical features:")
display(data.head())
```

```
Numerical columns to scale: ['Age', 'Glucose', 'Blood Pressure', 'BMI', 'Oxygen Saturation', 'LengthOfStay'.

DataFrame after Min-Max scaling of numerical features:
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smo
0	0.455696	0.391428	0.401144	0.431680	0.670348	0.277778	0.518390	0.524877	0.476872	
1	0.151899	0.171904	0.256540	0.378609	0.705592	0.055556	0.265915	0.342102	0.179515	
2	0.506329	0.251249	0.421191	0.302155	0.535714	0.055556	0.453891	0.423050	0.255507	
3	0.594937	NaN	0.369988	0.626271	0.683506	0.222222	0.388288	0.460844	0.400881	

```python
import numpy as np

# Identify categorical columns (excluding 'random_notes' and 'noise_col' for now as they might require speci
categorical_cols = data.select_dtypes(include=['object', 'category', 'bool']).columns.tolist()

# Exclude columns that are likely free text or not meant for one-hot encoding directly
# 'random_notes' is free text, 'noise_col' is numerical based on initial data head.
# 'Gender' and 'Medical Condition' are prime candidates for one-hot encoding.
if 'random_notes' in categorical_cols: categorical_cols.remove('random_notes')
if 'noise_col' in categorical_cols: categorical_cols.remove('noise_col') # Although noise_col seems numeric

print("Categorical columns to encode:", categorical_cols)

# Apply one-hot encoding to identified categorical columns
data = pd.get_dummies(data, columns=categorical_cols, drop_first=True, dtype=int)

print("\nDataFrame after one-hot encoding:")
display(data.head())
```

```
Categorical columns to encode: ['Gender', 'Medical Condition']

DataFrame after one-hot encoding:
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	HbA1c	Smoking	...	S H
0	46.0	137.04	135.27	28.90	96.04	6	231.88	210.56	7.61	0	...	
1	22.0	71.58	113.27	26.29	97.54	2	165.57	129.41	4.91	0	...	
2	50.0	95.24	138.32	22.53	90.31	2	214.94	165.35	5.60	0	...	
3	57.0	NaN	130.53	38.47	96.60	5	197.71	182.13	6.92	0	...	
4	66.0	95.15	178.17	31.12	94.90	4	259.53	115.85	5.98	0	...	

5 rows × 25 columns

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

- A structured dataset ready for HR analytics.

```python
import pandas as pd

# Load the California housing dataset
df = pd.read_csv('/content/sample_data/california_housing_train.csv')

# Display the first 5 rows of the DataFrame
display(df.head())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_inc
0	-114.31	34.19	15.0	5612.0	1283.0	1015.0	472.0	1.4
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0	463.0	1.8
2	-114.56	33.69	17.0	720.0	174.0	333.0	117.0	1.6
3	-114.57	33.64	14.0	1501.0	337.0	515.0	226.0	3.1
4	-114.57	33.57	20.0	1454.0	326.0	624.0	262.0	1.9

```python
display(df.describe())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	househol
count	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.0000
mean	-119.562108	35.625225	28.589353	2643.664412	539.410824	1429.573941	501.2219
std	2.005166	2.137340	12.586937	2179.947071	421.499452	1147.852959	384.5208
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.0000
25%	-121.790000	33.930000	18.000000	1462.000000	297.000000	790.000000	282.0000
50%	-118.490000	34.250000	29.000000	2127.000000	434.000000	1167.000000	409.0000
75%	-118.000000	37.720000	37.000000	3151.250000	648.250000	1721.000000	605.2500
max	-114.310000	41.950000	52.000000	37937.000000	6445.000000	35682.000000	6082.0000

50%	-118.490000	34.250000	29.000000	2127.000000	434.000000	1167.000000	409.000000
75%	-118.000000	37.720000	37.000000	3151.250000	648.250000	1721.000000	605.250000
max	-114.310000	41.950000	52.000000	37937.000000	6445.000000	35682.000000	6082.000000

```python
missing_values = df.isnull().sum()
print("Missing values in each column:")
display(missing_values[missing_values > 0])
```

```
Missing values in each column:
```

	0

dtype: int64

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan_amount, credit_score.
- Encode loan_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

import pandas as pd

data = pd.read_csv("loadapproval.csv")

print(data.head())

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:

- A machine-learning-ready loan dataset.

Code:

```
import pandas as pd
data = pd.read_csv("/content/archive (10).zip")
print(data.head())
```

```
   applicant_id  age  gender marital_status  annual_income  loan_amount  \
0             1   59    Male       Divorced         100073         7169
1             2   49    Male        Married         112197        23556
2             3   35    Male       Divorced          84429        27052
3             4   63  Female         Single         124195        11313
4             5   28  Female        Married          81627        13315

   credit_score  num_dependents  existing_loans_count employment_status  \
0           793               1                     1        Unemployed
1           789               0                     2          Employed
2           372               1                     4        Unemployed
3           808               3                     4     Self-employed
4           689               0                     1        Unemployed

   loan_approved
0              1
1              1
2              0
3              1
4              1
```

```
print('Missing values before imputation:')
display(data[['loan_amount', 'credit_score']].isnull().sum())

# Impute missing values in 'loan_amount' and 'credit_score' with their medians
data['loan_amount'] = data['loan_amount'].fillna(data['loan_amount'].median())
data['credit_score'] = data['credit_score'].fillna(data['credit_score'].median())

print('\nMissing values after imputation:')
display(data[['loan_amount', 'credit_score']].isnull().sum())
```

```
Missing values before imputation:
```

	0

```python
print('\nMissing values after imputation:')
display(data[['loan_amount', 'credit_score']].isnull().sum())
```

```
Missing values before imputation:
```

	0
loan_amount	0
credit_score	0

dtype: int64

```
Missing values after imputation:
```

	0
loan_amount	0
credit_score	0

dtype: int64

```python
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Identify numerical features to scale (excluding 'applicant_id' and 'loan_approved')
numerical_cols = ['age', 'annual_income', 'loan_amount', 'credit_score', 'num_dependents', 'existing_lc

# Initialize StandardScaler
scaler = StandardScaler()

# Apply scaling to the numerical columns
data[numerical_cols] = scaler.fit_transform(data[numerical_cols])

print("Scaled numerical features (first 5 rows):")
display(data[numerical_cols].head())
```

```
Scaled numerical features (first 5 rows):
```

	age	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count
0	1.307840	0.482301	-1.566427	1.391620	-0.689042	-0.737500
1	0.514489	0.805362	-0.287825	1.365954	-1.394305	-0.029726
2	-0.596204	0.065444	-0.015048	-1.309808	-0.689042	1.385820
3	1.625181	1.125066	-1.243090	1.487871	0.721484	1.385820
4	-1.151550	-0.009219	-1.086883	0.724284	-1.394305	-0.737500

```python
# Define features (X) and target (y)
X = data.drop(['applicant_id', 'loan_approved'], axis=1)
y = data['loan_approved']

# For simplicity, let's one-hot encode categorical features if any, before splitting.
# Check for categorical columns
categorical_cols = X.select_dtypes(include=['object']).columns

if not categorical_cols.empty:
    X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
    print("Features after one-hot encoding categorical variables (first 5 rows):")
    display(X.head())
else:
    print("No categorical columns found for one-hot encoding.")

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nX_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")
```

```
Features after one-hot encoding categorical variables (first 5 rows):
```

	age	annual_income	loan_amount	credit_score	num_dependents	existing_loans_count	gender_Male	mar
0	1.307840	0.482301	-1.566427	1.391620	-0.689042	-0.737500	True	
1	0.514489	0.805362	-0.287825	1.365954	-1.394305	-0.029726	True	
2	-0.596204	0.065444	-0.015048	-1.309808	-0.689042	1.385820	True	
3	1.625181	1.125066	-1.243090	1.487871	0.721484	1.385820	False	
4	-1.151550	-0.009219	-1.086883	0.724284	-1.394305	-0.737500	False	

```
X_train shape: (800, 11)
X_test shape: (200, 11)
y_train shape: (800,)
y_test shape: (200,)
```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- Remove duplicate user profiles.
- Handle missing values
- Encode categorical columns

Dataset link:

https://www.kaggle.com/datasets/medaxone/top-100-social- media -profiles

Expected Output:

- A cleaned dataset for engagement prediction.

```python
import pandas as pd
import zipfile
import os

zip_file_path = "/content/archive (13).zip"
extract_dir = "/content/extracted_data"

# Create a directory to extract files into
os.makedirs(extract_dir, exist_ok=True)

# Unzip the file
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_dir)

# Find the CSV file within the extracted directory (assuming there's one CSV)
csv_files = [f for f in os.listdir(extract_dir) if f.endswith('.csv')]

if csv_files:
    # Assuming the first CSV file found is the correct one
    csv_file_name = csv_files[0]
    # Try reading the CSV with 'latin1' encoding to resolve UnicodeDecodeError
    data = pd.read_csv(os.path.join(extract_dir, csv_file_name), encoding='latin1')
    print(data.head())
else:
    print(f"No CSV file found in the extracted directory: {extract_dir}")
```

```
   N             PROFILE    FOLLOWERS   POSTS    Platform
0  1           Instagram   539446645.0  7202.0  Instagram
1  2   Cristiano Ronaldo   473864939.0  3338.0  Instagram
2  3  Kylie <U+0001F90D>   364542529.0  6935.0  Instagram
3  4           Leo Messi   355790796.0   890.0  Instagram
4  5        Selena Gomez   341579063.0  1828.0  Instagram
```

```python
initial_rows = data.shape[0]
data.drop_duplicates(subset=['PROFILE'], inplace=True)
removed_rows = initial_rows - data.shape[0]
print(f"Removed {removed_rows} duplicate user profiles. New dataset size: {data.shape[0]} rows.")
```

```
Removed 35 duplicate user profiles. New dataset size: 465 rows.
```

```python
print('Missing values before handling:')
print(data.isnull().sum())

# Identify numerical and categorical columns for imputation
numerical_cols = data.select_dtypes(include=['number']).columns
categorical_cols = data.select_dtypes(include=['object']).columns

# Impute missing numerical values with the median
for col in numerical_cols:
    if data[col].isnull().any():
        median_val = data[col].median()
        data[col] = data[col].fillna(median_val)
        print(f"Imputed missing values in numerical column '{col}' with median: {median_val}")

# Impute missing categorical values with the mode
for col in categorical_cols:
    if data[col].isnull().any():
        mode_val = data[col].mode()[0] # .mode() can return multiple values, take the first
        data[col] = data[col].fillna(mode_val)
        print(f"Imputed missing values in categorical column '{col}' with mode: {mode_val}")

print('\nMissing values after handling:')
print(data.isnull().sum())
```

```
Missing values before handling:
```

```
Missing values before handling:
N                                  0
FOLLOWERS                          0
POSTS                              0
PROFILE_9GAG: Go Fun The World     0
PROFILE_<U+0001F451>               0
                                  ..
PROFILE_zayn                       0
Platform_TikTok                    0
Platform_Twitch                    0
Platform_Twitter                   0
Platform_YouTube                   0
Length: 471, dtype: int64

Missing values after handling:
N                                  0
FOLLOWERS                          0
POSTS                              0
PROFILE_9GAG: Go Fun The World     0
PROFILE_<U+0001F451>               0
                                  ..
PROFILE_zayn                       0
Platform_TikTok                    0
Platform_Twitch                    0
Platform_Twitter                   0
Platform_YouTube                   0
Length: 471, dtype: int64
```

```
print("Categorical columns before encoding:")
print(data.select_dtypes(include=['object']).columns.tolist())

# Perform one-hot encoding on categorical columns
data = pd.get_dummies(data, columns=categorical_cols, drop_first=True)

print("\nDataFrame head after one-hot encoding:")
display(data.head())
print("\nDataFrame info after one-hot encoding:")
data.info()
```

```
orical columns before encoding:
FILE', 'Platform']

ame head after one-hot encoding:
```

FOLLOWERS	POSTS	PROFILE_9GAG: Go Fun The World	PROFILE_<U+0001F451>	PROFILE_<U+0001F49B>julia menu garcia<U+0001F49B>	PROFILE_<U+0410>nokhina Liza
539446645.0	7202.0	False	False	False	False
473864939.0	3338.0	False	False	False	False
364542529.0	6935.0	False	False	False	False
355790796.0	890.0	False	False	False	False
341579063.0	1828.0	False	False	False	False

× 471 columns

```
ame info after one-hot encoding:
s 'pandas.core.frame.DataFrame'>
: 465 entries, 0 to 499
ns: 471 entries, N to Platform_YouTube
```

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.
- Handle missing values in temperature, humidity.

• Create new features (season, week_number).

• Normalize rainfall values.

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:

• A processed dataset ready for forecasting models

Code:

```python
import pandas as pd

df_california_housing = pd.read_csv('/content/sample_data/california_housing_train.csv')
display(df_california_housing.head())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_incor
0	-114.31	34.19	15.0	5612.0	1283.0	1015.0	472.0	1.49
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0	463.0	1.82
2	-114.56	33.69	17.0	720.0	174.0	333.0	117.0	1.65
3	-114.57	33.64	14.0	1501.0	337.0	515.0	226.0	3.19
4	-114.57	33.57	20.0	1454.0	326.0	624.0	262.0	1.92

```python
dataframes_to_check = {
    'X': X,
    'X_test': X_test,
    'X_train': X_train,
    'data': data,
    'df': df,
    'df_dummy': df_dummy,
    'dummy_planets_df': dummy_planets_df
}

found_columns = False
for df_name, dataframe in dataframes_to_check.items():
    if 'temperature' in dataframe.columns:
        print(f"DataFrame '{df_name}' contains a 'temperature' column.")
        found_columns = True
    if 'humidity' in dataframe.columns:
        print(f"DataFrame '{df_name}' contains a 'humidity' column.")
        found_columns = True

if not found_columns:
```

```python
dataframes_to_check = {
    'X': X,
    'X_test': X_test,
    'X_train': X_train,
    'data': data,
    'df': df,
    'df_dummy': df_dummy,
    'dummy_planets_df': dummy_planets_df
}

found_columns = False
for df_name, dataframe in dataframes_to_check.items():
    if 'temperature' in dataframe.columns:
        print(f"DataFrame '{df_name}' contains a 'temperature' column.")
        found_columns = True
    if 'humidity' in dataframe.columns:
        print(f"DataFrame '{df_name}' contains a 'humidity' column.")
        found_columns = True

if not found_columns:
    print("No 'temperature' or 'humidity' columns found in the other loaded DataFrames.")
```

```
No 'temperature' or 'humidity' columns found in the other loaded DataFrames.
```

```python
df_california_housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
```

Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.
- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:

- A dataset suitable for recommendation systems.

Code:

```python
import pandas as pd
data = pd.read_csv("/content/archive (15).zip")
print(data.head())
```

```
   Rank                    Title                     Genre  \
0     1  Guardians of the Galaxy   Action,Adventure,Sci-Fi
1     2               Prometheus  Adventure,Mystery,Sci-Fi
2     3                    Split           Horror,Thriller
3     4                     Sing   Animation,Comedy,Family
4     5            Suicide Squad  Action,Adventure,Fantasy

                                        Description             Director  \
0  A group of intergalactic criminals are forced ...            James Gunn
1  Following clues to the origin of mankind, a te...          Ridley Scott
2  Three girls are kidnapped by a man with a diag...    M. Night Shyamalan
3  In a city of humanoid animals, a hustling thea...  Christophe Lourdelet
4  A secret government agency recruits some of th...            David Ayer

                                            Actors  Year  Runtime (Minutes)  \
0  Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...  2014                121
1  Noomi Rapace, Logan Marshall-Green, Michael Fa...  2012                124
2  James McAvoy, Anya Taylor-Joy, Haley Lu Richar...  2016                117
3  Matthew McConaughey,Reese Witherspoon, Seth Ma...  2016                108
4  Will Smith, Jared Leto, Margot Robbie, Viola D...  2016                123

   Rating   Votes  Revenue (Millions)  Metascore
0     8.1  757074              333.13       76.0
1     7.0  485820              126.46       65.0
2     7.3  157606              138.12       62.0
3     7.2   60545              270.32       59.0
4     6.2  393727              325.02       40.0
```

```python
# Remove duplicate movie entries
initial_rows = data.shape[0]
data.drop_duplicates(inplace=True)
removed_rows = initial_rows - data.shape[0]
print(f"Removed {removed_rows} duplicate rows. Remaining rows: {data.shape[0]}")

# Display the head of the DataFrame after removing duplicates
print("\nDataFrame after removing duplicates:")
display(data.head())
```

```
Removed 0 duplicate rows. Remaining rows: 1000

DataFrame after removing duplicates:
```

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12
3	4	Sing	In a city of humanoid animals, a hustling thea	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32

```python
from sklearn.preprocessing import MinMaxScaler

# Initialize MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))

# Reshape the 'Rating' column for scaling (MinMaxScaler expects a 2D array)
data['Rating_Scaled'] = scaler.fit_transform(data[['Rating']])

# Display the DataFrame with the new scaled 'Rating' column
print("\nDataFrame after scaling 'Rating' column:")
display(data[['Rating', 'Rating_Scaled']].head())

# Verify the scaling range
print(f"Min of Scaled Rating: {data['Rating_Scaled'].min()}")
print(f"Max of Scaled Rating: {data['Rating_Scaled'].max()}")
```

```
DataFrame after scaling 'Rating' column:
```

	Rating	Rating_Scaled
0	8.1	0.873239
1	7.0	0.718310
2	7.3	0.760563
3	7.2	0.746479
4	6.2	0.605634

```
Min of Scaled Rating: 0.0
Max of Scaled Rating: 1.0
```

```python
import pandas as pd
data = pd.read_csv("/content/archive (15).zip")
print(data.head())

# Handle missing values in 'Rating' (numerical column) with the median
median_rating = data['Rating'].median()
data['Rating'].fillna(median_rating, inplace=True)

# Handle missing values in 'Genre' (categorical column) with the mode
# If there are multiple modes, .mode()[0] picks the first one.
mode_genre = data['Genre'].mode()[0]
data['Genre'].fillna(mode_genre, inplace=True)

# Check for any remaining missing values in these columns
print(f"\nMissing values after imputation in 'Rating': {data['Rating'].isnull().sum()}")
print(f"Missing values after imputation in 'Genre': {data['Genre'].isnull().sum()}")
```

```
   Rank                    Title                     Genre  \
0     1  Guardians of the Galaxy   Action,Adventure,Sci-Fi
1     2              Prometheus  Adventure,Mystery,Sci-Fi
2     3                   Split           Horror,Thriller
3     4                    Sing   Animation,Comedy,Family
4     5            Suicide Squad  Action,Adventure,Fantasy

                                       Description              Director  \
0  A group of intergalactic criminals are forced ...            James Gunn
1  Following clues to the origin of mankind, a te...           Ridley Scott
2  Three girls are kidnapped by a man with a diag...     M. Night Shyamalan
3  In a city of humanoid animals, a hustling thea...   Christophe Lourdelet
4  A secret government agency recruits some of th...            David Ayer

                                          Actors  Year  Runtime (Minutes)  \
0  Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...  2014                121
1  Noomi Rapace, Logan Marshall-Green, Michael Fa...  2012                124
2  James McAvoy, Anya Taylor-Joy, Haley Lu Richar...  2016                117
3  Matthew McConaughey,Reese Witherspoon, Seth Ma...  2016                108
4  Will Smith, Jared Leto, Margot Robbie, Viola D...  2016                123

   Rating   Votes  Revenue (Millions)  Metascore
0     8.1  757074              333.13       76.0
1     7.0  485820              126.46       65.0
```

```python
from sklearn.preprocessing import MultiLabelBinarizer

# Split the genres into lists of individual genres
data['Genre_List'] = data['Genre'].apply(lambda x: x.split(','))

# Initialize MultiLabelBinarizer
mlb = MultiLabelBinarizer()

# Fit and transform the 'Genre_List' column
genre_encoded = pd.DataFrame(mlb.fit_transform(data['Genre_List']),
                             columns=mlb.classes_,
                             index=data.index)

# Concatenate the new encoded genre columns with the original DataFrame
data = pd.concat([data, genre_encoded], axis=1)

# Drop the original 'Genre' and 'Genre_List' columns if desired
data.drop(columns=['Genre', 'Genre_List'], inplace=True)

# Display the updated DataFrame with encoded genres
print("\nDataFrame after genre encoding:")
display(data.head())
```

DataFrame after genre encoding:

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Horror	Music	Musical
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	0	0	0
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	...	0	0	0
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	1	0	0
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	0	0	0